

Manipulation des contrôleurs

Intérêt des contrôleurs

Intérêt des contrôleurs

- Au lieu de définir toute votre logique de gestion des requêtes comme des fermetures dans vos fichiers de routage, vous pouvez organiser ce comportement à l'aide de classes "**contrôleur**".
- Les contrôleurs peuvent regrouper la logique de gestion des demandes associées dans une seule classe. Par exemple, une classe *UserController* peut gérer toutes les demandes entrantes liées aux utilisateurs, y compris l'affichage, la création, la mise à jour et la suppression d'utilisateurs. Par défaut, les contrôleurs sont stockés dans le répertoire **app/Http/Controllers**.

Implémentation de contrôleur

Contrôleurs de base

- Examinons un exemple de contrôleur de base. Notez que le contrôleur étend la classe de contrôleur de base incluse avec Laravel **App\Http\Controllers\Controller**

```
<?php
namespace App\Http\Controllers;
use App\Http\Controllers\Controller;
use App\Models\User;
class UserController extends Controller
{
    /**
     * Show the profile for a given user.
     *
     * @param int $id
     * @return \Illuminate\View\View
     */
    public function show($id)
```

Contrôleurs de base

- Vous pouvez définir une route vers cette méthode de contrôleur comme suit :

```
use App\Http\Controllers\UserController;  
  
Route::get('/user/{id}', [UserController::class, 'show']);
```

- Lorsqu'une requête entrante correspond à l'URI de route spécifié, la méthode de la classe **App\Http\Controllers\UserController** est appelée **show** et les paramètres de route sont transmis à la méthode.

Contrôleurs à simple action

- Si une action de contrôleur est particulièrement complexe, vous trouverez peut-être pratique de dédier une classe de contrôleur entière à cette action unique. Pour ce faire, vous pouvez définir une seule méthode `__invoke` dans le contrôleur :

```
<?php
namespace App\Http\Controllers;
use App\Http\Controllers\Controller;
use App\Models\User;
class ProvisionServer extends Controller
{
    /**
     * Provision a new web server.
     *
     * @return \Illuminate\Http\Response
     */
}
```

Contrôleurs à simple action

- Lors de l'enregistrement d'itinéraires pour des contrôleurs à action unique, vous n'avez pas besoin de spécifier une méthode de contrôleur. Au lieu de cela, vous pouvez simplement transmettre le nom du contrôleur au routeur :

```
use App\Http\Controllers\ProvisionServer;  
  
Route::post('/server', ProvisionServer::class);
```

- Vous pouvez générer un contrôleur invocable en utilisant l'option --invokable de la commande Artisan make:controller :

```
php artisan make:controller ProvisionServer --invokable
```


Contrôleur Middleware

Contrôleurs à simple action

- Un middleware peut être affecté aux routes du contrôleur dans vos fichiers de routes :

```
Route::get('profile', [UserController::class, 'show'])->middleware('auth');
```

- Ou, vous trouverez peut-être pratique de spécifier un middleware dans le constructeur de votre contrôleur. En utilisant la méthode `middleware` dans le constructeur de votre contrôleur, vous pouvez affecter un middleware aux actions du contrôleur :

```
class UserController extends Controller
{
    /**
     * Instantiate a new controller instance.
     */
}
```

Contrôleurs à simple action

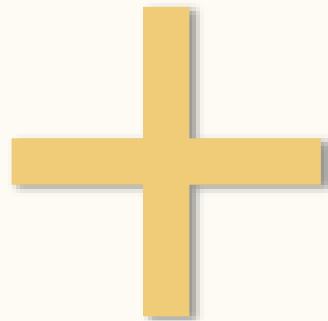
- Les contrôleurs vous permettent également d'enregistrer un middleware à l'aide d'une fermeture. Cela fournit un moyen pratique de définir un middleware en ligne pour un seul contrôleur sans définir une classe complète de middleware :

```
$this->middleware(function ($request, $next) {  
    return $next($request);  
});
```

Contrôleur de ressources

Contrôleur de ressources

- Si vous considérez chaque modèle Eloquent de votre application comme une "ressource", il est courant d'effectuer les mêmes ensembles d'actions sur chaque ressource de votre application. Par exemple, imaginez que votre application contient un modèle **Photo** et un modèle **Movie**. Il est probable que les utilisateurs puissent créer, lire, mettre à jour ou supprimer ces ressources.



Contrôleur de ressources

- En raison de ce cas d'utilisation courant, le routage des ressources Laravel attribue les routes typiques de création, lecture, mise à jour et suppression ("CRUD") à un contrôleur avec une seule ligne de code. Pour commencer, nous pouvons utiliser l'option `make:controller` de la commande Artisan `--resource` pour créer rapidement un contrôleur pour gérer ces actions :

```
php artisan make:controller PhotoController --resource
```

Contrôleur de ressources

```
php artisan make:controller PhotoController --resource
```

- Cette commande générera un contrôleur à **app/Http/Controllers/PhotoController.php**. Le contrôleur contiendra une méthode pour chacune des opérations de ressources disponibles. Ensuite, vous pouvez enregistrer une route de ressources qui pointe vers le contrôleur :

```
use App\Http\Controllers\PhotoController;
```

```
Route::resource('photos', PhotoController::class);
```

Contrôleur de ressources

```
use App\Http\Controllers\PhotoController;
```

```
Route::resource('photos', PhotoController::class);
```

- Cette déclaration de route unique crée plusieurs routes pour gérer diverses actions sur la ressource. Le contrôleur généré aura déjà des méthodes stub pour chacune de ces actions. N'oubliez pas que vous pouvez toujours obtenir un aperçu rapide des routes de votre application en exécutant la commande `Artisan route:list`.

Contrôleur de ressources

- Vous pouvez même enregistrer plusieurs contrôleurs de ressources à la fois en passant un tableau à la méthode resources :

```
Route::resources([  
    'photos' => PhotoController::class,  
    'posts' => PostController::class,  
]);
```

Actions gérées par le contrôleur de ressources

Verbe	URI	Action	Nom de la route
GET	/photos	index	photos.index
GET	/photos/create	create	photos.create
POST	/photos	store	photos.store
GET	/photos/{photo}	show	photos.show
GET	/photos/{photo}/edit	edit	photos.edit
PUT/PATCH	/photos/{photo}	update	photos.update
DELETE	/photos/{photo}	destroy	photos.destroy

Personnalisation du comportement du modèle manquant

- En règle générale, une réponse HTTP 404 est générée si un modèle de ressource implicitement lié n'est pas trouvé. Cependant, vous pouvez personnaliser ce comportement en appelant la méthode `missing` lors de la définition de votre route de ressource. La méthode `missing` accepte une fermeture qui sera invoquée si un modèle lié implicitement ne peut être trouvé pour aucune des routes de la ressource :

```
use App\Http\Controllers\PhotoController;  
use Illuminate\Http\Request;  
use Illuminate\Support\Facades\Redirect;  
  
Route::resource('photos', PhotoController::class)
```

Spécification du modèle de ressource

- Si vous utilisez la liaison de modèle de route et que vous souhaitez que les méthodes du contrôleur de ressources indiquent le type d'une instance de modèle, vous pouvez utiliser l'option **--model** lors de la génération du contrôleur :

```
php artisan make:controller PhotoController --model=Photo --resource
```

Génération de demandes de formulaire

- Vous pouvez fournir l'option **--requests** lors de la génération d'un contrôleur de ressources pour demander à Artisan de générer des classes de requête de formulaire pour les méthodes de stockage et de mise à jour du contrôleur :

```
php artisan make:controller PhotoController --model=Photo  
--resource --requests
```

Routes de ressources partielles

- Lors de la déclaration d'une route de ressources, vous pouvez spécifier un sous-ensemble d'actions que le contrôleur doit gérer au lieu de l'ensemble complet d'actions par défaut :

```
use App\Http\Controllers\PhotoController;
```

```
Route::resource('photos', PhotoController::class)->only([  
    'index', 'show'  
]);
```

```
Route::resource('photos', PhotoController::class)->except([  
    'create', 'store', 'update', 'destroy'  
]);
```

Itinéraires de ressources d'API

- Lors de la déclaration des routes de ressources qui seront consommées par les API, vous souhaitez généralement exclure les routes qui présentent des modèles HTML tels que create et edit. Pour plus de commodité, vous pouvez utiliser la méthode `apiResource` pour exclure automatiquement ces deux routes :

```
use App\Http\Controllers\PhotoController;  
Route::apiResource('photos', PhotoController::class);
```

Itinéraires de ressources d'API

- Vous pouvez enregistrer plusieurs contrôleurs de ressources d'API à la fois en passant un tableau à la méthode `apiResources` :

```
use App\Http\Controllers\PhotoController;  
use App\Http\Controllers\PostController;  
  
Route::apiResources([  
    'photos' => PhotoController::class,  
    'posts' => PostController::class,  
]);
```


Contrôleur de ressources

- Pour générer rapidement un contrôleur de ressources d'API qui n'inclut pas les méthodes **create** ou **edit**, utilisez le commutateur **--api** lors de l'exécution de la commande : **make:controller**

```
php artisan make:controller PhotoController --api
```

Ressources imbriquées

- Parfois, vous devrez peut-être définir des itinéraires vers une ressource imbriquée. Par exemple, une ressource photo peut avoir plusieurs commentaires qui peuvent être joints à la photo. Pour imbriquer les contrôleurs de ressources, vous pouvez utiliser la notation "point" dans votre déclaration de route :

```
use App\Http\Controllers\PhotoCommentController;  
Route::resource('photos.comments',  
PhotoCommentController::class);
```

Ressources imbriquées

- Cette route enregistrera une ressource imbriquée accessible avec des URI comme suit :

`/photos/{photo}/comments/{comment}`

Nidification à faible profondeur

- Souvent, il n'est pas tout à fait nécessaire d'avoir à la fois les identifiants parent et enfant dans un URI puisque l'identifiant enfant est déjà un identifiant unique. Lorsque vous utilisez des identifiants uniques tels que des clés primaires à incrémentation automatique pour identifier vos modèles dans des segments d'URI, vous pouvez choisir d'utiliser une "imbrication peu profonde":

```
use App\Http\Controllers\CommentController;
```

```
Route::resource('photos.comments', CommentController::class)->shallow();
```

Cette définition de route définira les routes suivantes :

Verbe	URI	Action	Nom de la route
GET	/photos/{photo}/comments	index	photos.comments.index
GET	/photos/{photo}/comments/create	create	photos.comments.create
POST	/photos/{photo}/comments	store	photos.comments.store
GET	/comments/{comment}	show	comments.show
GET	/comments/{comment}/edit	edit	comments.edit
PUT/PATCH	/comments/{comment}	update	comments.update
DELETE	/comments/{comment}	destroy	comments.destroy

Nommer les routes de ressources

- Par défaut, toutes les actions du contrôleur de ressources ont un nom de route ; cependant, vous pouvez remplacer ces noms en transmettant un tableau `names` avec les noms de route souhaités :

```
use App\Http\Controllers\PhotoController;

Route::resource('photos', PhotoController::class)->names([
    'create' => 'photos.build'
]);
```

Injection de dépendances

Injection de constructeur

- Le conteneur de service Laravel est utilisé pour résoudre tous les contrôleurs Laravel. En conséquence, vous pouvez indiquer toutes les dépendances dont votre contrôleur peut avoir besoin dans son constructeur. Les dépendances déclarées seront automatiquement résolues et injectées dans l'instance du contrôleur :


```
<?php
namespace App\Http\Controllers;
use App\Repositories\UserRepository;
class UserController extends Controller
{/**
 * The user repository instance.
 */
protected $users;
/**
 * Create a new controller instance.
 * @param \App\Repositories\UserRepository $users
 * @return void
 */
public function __construct(UserRepository $users)
{
    $this->users = $users;
}
}
```

Injection par méthode

- En plus de l'injection par constructeur, vous pouvez également créer des dépendances d'indication de type sur les méthodes de votre contrôleur. Un cas d'utilisation courant pour l'injection de méthode consiste à injecter l'instance `Illuminate\Http\Request` dans les méthodes de votre contrôleur :

```
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;
class UserController extends Controller
{
/**
 * Store a new user.
 * @param \Illuminate\Http\Request $request
 * @return \Illuminate\Http\Response
 */
public function store(Request $request)
{
$name = $request->name;
//
}
}
```

Injection de méthode

- Si votre méthode de contrôleur attend également une entrée d'un paramètre de route, lister vos arguments de route après vos autres dépendances. Par exemple, si votre route est définie comme ceci :

```
use App\Http\Controllers\UserController;  
  
Route::put('/user/{id}', [UserController::class, 'update']);
```

Injection de méthode

- Vous pouvez toujours taper le `Illuminate\Http\Request` et accéder à votre paramètre `id` en définissant votre méthode de contrôleur comme suit :

```
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;
class UserController extends Controller
{/**
 * Update the given user.
 *
 * @param \Illuminate\Http\Request $request
 * @param string $id
 * @return \Illuminate\Http\Response
 */
public function update(Request $request, $id)
{
//
}
}
```